For high-frequency trading (HFT) inference acceleration on Xilinx FPGAs, the optimal approach combines **AMD Alveo UL3524** cards (achieving **13.9ns** STAC-T0 tick-to-trade latency) with **decision tree ensembles (XGBoost/LightGBM)** for sub-microsecond ML inference, or **quantized neural networks** via the **FINN/hls4ml** flow for more complex signal generation. The complete implementation follows a five-stage pipelined architecture: **network ingress → market data parsing → order book maintenance → ML signal evaluation → order transmission**, with critical HLS pragmas (`#pragma HLS PIPELINE II=1`, `#pragma HLS DATAFLOW`, `#pragma HLS ARRAY_PARTITION`) enabling deterministic single-cycle throughput.

---

# Implementing Machine Learning Algorithms on Xilinx FPGA for High-Frequency Trading Inference Acceleration

## 1. Executive Summary: The Case for FPGA-Based ML Inference in HFT

### 1.1 Why FPGA Acceleration for HFT ML

High-frequency trading operates in a domain where latency is the primary competitive differentiator, and the gap between a profitable trade and a missed opportunity is measured in nanoseconds. Traditional software-based trading systems execute on CPUs with latencies ranging from **10 to 50 microseconds** for the complete tick-to-trade loop, while FPGA-accelerated systems have demonstrated end-to-end latencies as low as **13.9 nanoseconds** in independently verified STAC-T0 benchmarks (A Team Insight) . This **three-order-of-magnitude improvement** fundamentally changes the economics of algorithmic trading: where a CPU system might capture **1 in 1000 arbitrage opportunities**, an FPGA system captures **950 in 1000**, translating to revenue differences of millions of dollars per day (Medium) .

Field-programmable gate arrays deliver this performance through several architectural advantages that are uniquely suited to both the deterministic requirements of HFT and the computational patterns of machine learning inference. Unlike CPUs, which execute instructions sequentially through complex pipelines subject to cache misses, branch mispredictions, and context switches, FPGAs implement algorithms directly as spatial hardware circuits. Every operation — from network packet parsing through ML model inference to order message construction — executes in dedicated logic with **guaranteed, cycle-accurate timing** that does not vary with system load or data patterns (linaro.org) . This determinism is critical in HFT, where the **99th-percentile latency** matters as much as the median; FPGA implementations eliminate the tail latency spikes caused by OS scheduling, garbage collection, and cache contention that plague software systems (xelera.io) .

For machine learning specifically, FPGAs offer a convergence of attributes that no other platform matches. They support **custom data precision** — trading strategies rarely need FP32 precision, and FPGAs natively implement INT8, INT4, or even binary/ternary arithmetic with proportional reductions in resource usage and latency (xilinx.github.io) . They enable **true dataflow architectures** where each neural network layer or decision tree node is a physically distinct pipeline stage, processing data at the rate of one inference per clock cycle after initial fill (indico.global) . And they provide **massive on-chip memory bandwidth** through BRAM and UltraRAM resources, keeping model weights and intermediate activations local to the computation and avoiding the DRAM access penalties that dominate GPU and CPU inference latency (arXiv.org) .

### 1.2 Performance Landscape: FPGA vs. GPU vs. CPU for HFT ML

The performance comparison between hardware platforms for HFT ML inference reveals stark differences across latency, determinism, power efficiency, and development complexity. CPUs remain the default for strategy prototyping and non-latency-sensitive execution, but their **sequential execution model** and **OS-mediated resource sharing** introduce variability that makes them unsuitable for the critical path of tick-to-trade pipelines. GPUs excel at throughput-oriented batch inference and model training, but their **SIMT architecture** requires batching to achieve efficiency, fundamentally conflicting with the batch-size-1, microsecond-latency requirements of HFT signal generation (linaro.org) .

| Platform | Min Latency (batch=1) | Determinism | Power Efficiency | ML Throughput | Best Use Case in HFT |
|---|---|---|---|---|---|
| **CPU (Xeon)** | 10–50 s (Medium) | Poor (cache, scheduling) | Baseline | Low | Strategy backtesting, risk reporting |
| **GPU (A100)** | 50–200 s (Github) | Poor (kernel launch, memory) | 3–4× worse than FPGA (Medium) | Very High (batched) | Model training, overnight batch analytics |
| **FPGA (Alveo UL3524)** | **13.9 ns** (network I/O) (A Team Insight) | **Excellent** (cycle-accurate) | **3–4× better than GPU** (Medium) | High (streaming) | **Tick-to-trade ML inference** |
| **FPGA + HBM (U280)** | ~100 ns (full pipeline) (www.semidesignjobs.com) | Excellent | Good | Very High | Multi-model ensemble inference |

The table above captures the essential trade-offs. FPGAs sacrifice the programmability and ecosystem maturity of CPUs and the raw compute throughput of GPUs to deliver the one metric that matters most in HFT: **deterministic, sub-microsecond latency for single-request inference**. The AMD Alveo UL3524 accelerator card, purpose-built for ultra-low-latency trading, achieves **<3ns transceiver latency** and has been validated at **13.9ns actionable latency** in the STAC-T0 benchmark — the lowest ever recorded (AMD) . For ML inference specifically, commercial solutions like Xelera Silva deliver **1.0–1.4 s** inference latency for gradient-boosted tree models with **1.179 s at p99**, eliminating the latency spikes that erode trading profits (xelera.io) .

### 1.3 Key Metrics: Latency, Throughput, Determinism, and Power

The design of an FPGA-based ML inference system for HFT requires careful balancing of four interdependent metrics, each with direct implications for trading profitability and operational feasibility. **Latency** in this context refers specifically to the inference latency — the time from when a feature vector (derived from order book state) is presented to the ML accelerator until a prediction (buy/sell/hold signal) is produced. For the tick-to-trade use case, this must be in the **sub-microsecond to nanosecond range**, as every additional nanosecond of delay reduces the probability of order fill at the desired price (www.semidesignjobs.com) .

**Determinism** — the variance in latency across repeated executions — is equally critical. HFT strategies rely on precise timing guarantees; a system with 100ns average latency but 5 s tail latency is far less valuable than one with 200ns latency and 1ns variance. FPGA implementations achieve determinism by eliminating software layers entirely: the ML inference executes as a fixed hardware pipeline with no scheduling decisions, memory allocation, or conditional execution paths that could introduce jitter

(linaro.org) . The Exegy/AMD STAC-T0 benchmark achieved **200 picoseconds of jitter** — ten times lower than previous records — by implementing the entire critical path as asynchronous combinational logic within the FPGA fabric (A Team Insight) .

**Throughput** requirements in HFT are characterized by high message rates during market bursts rather than sustained average loads. A typical equities market data feed generates **8.5 million messages per second** during peak hours, with individual message processing requiring sub-microsecond handling (Medium) . FPGA architectures naturally handle this through **pipeline parallelism**: while one market event is in the ML inference stage, the next is being parsed, and the previous is in risk management, enabling sustained throughput of millions of inferences per second without latency degradation (www.semidesignjobs.com) .

**Power efficiency** directly affects data center operational costs and cooling capacity. FPGAs consume **3–4× less power than GPUs** achieving equivalent inference throughput, a consequence of their spatial execution model that avoids the data movement overheads inherent in GPU memory hierarchies (Medium) . The Alveo UL3524 operates at **125W TDP** with passive cooling, while delivering inference performance that would require multiple 300W+ GPUs (AMD) . This efficiency enables denser deployment in co-located exchange data centers where rack space and power are strictly constrained.

## 2. Xilinx FPGA Hardware Platforms for HFT ML Acceleration

### 2.1 AMD Alveo UL3524: The Premier HFT Accelerator

The AMD Alveo UL3524 represents the current state-of-the-art in FPGA-based trading acceleration, purpose-built from the silicon level for ultra-low-latency financial applications. Unlike general-purpose FPGA cards that prioritize compute density or memory capacity, the UL3524 optimizes every architectural element for the specific workload of tick-to-trade processing: minimal transceiver latency, deterministic network protocol handling, and sufficient programmable logic for custom trading algorithms (AMD) .

The card is built around a custom **Virtex UltraScale+ FPGA** fabricated on TSMC's 16nm process, featuring **787K LUTs**, **1.72M registers**, and **1,680 DSP48E2 slices** — providing ample resources for complex ML model implementations while leaving significant headroom for the network stack, order book, and risk management logic that must coexist on the same device (infinipoint.ai) . The memory subsystem includes **256Mb of embedded memory** (76Mb BRAM + 180Mb UltraRAM), enabling on-chip storage of substantial model weights without recourse to off-chip DRAM, plus **16GB of DDR4** and **72MB of QDRII+ SRAM** for larger data structures (infinipoint.ai) .

The defining feature of the UL3524 is its **GTF ultra-low-latency transceiver architecture**, which achieves **<3ns transceiver latency** — a **7× improvement** over previous-generation GTY transceivers (AMD) . These 64 GTF transceivers (operating at up to 28.21 Gb/s) are paired with hardened MAC/PCS blocks that process Ethernet framing directly in the FPGA fabric, eliminating the soft-logic overhead that previously consumed hundreds of nanoseconds in the network stack (A Team Insight) . The card presents **4× QSFP-DD ports** supporting 32×10/25G connections, enabling direct connectivity to exchange multicast feeds without intermediate switching (infinipoint.ai) .

For ML inference specifically, AMD explicitly supports the **FINN open-source framework** on the UL3524, enabling developers to train quantized neural networks in PyTorch, compile them to optimized FPGA dataflow architectures, and integrate them directly into the trading datapath (AMD) . This PyTorch-to-bitstream flow, combined with traditional Vivado RTL development, gives trading firms flexibility to deploy everything from simple decision tree classifiers to complex deep learning models on the

same hardware platform.

### 2.2 Alveo U250/U280: High-Bandwidth Inference Cards

For HFT strategies requiring more complex ML models — particularly multi-model ensembles, deep neural networks for market microstructure prediction, or simultaneous inference across multiple instruments — the Alveo U250 and U280 cards provide substantially higher compute and memory resources at the cost of increased latency relative to the UL3524. The **U250** features a **Virtex UltraScale+ VU13P FPGA** with **1.73M LUTs**, **6,840 DSP slices**, and **64GB of DDR4**, while the **U280** adds **8GB of HBM2** providing **460 GB/s** of memory bandwidth — critical for models with large weight footprints that exceed on-chip BRAM/URAM capacity (VMware Blogs) .

The HBM2 on the U280 is particularly relevant for ML inference scenarios involving large models or high-dimensional feature vectors. While the UL3524's on-chip memory suffices for compact decision trees and small neural networks, strategies employing **CNN-LSTM hybrids** or **Transformer-based market encoders** may require hundreds of megabytes of weight storage (ntu.edu.tw) . The U280's HBM2 enables these models to reside in high-bandwidth memory with access latencies of ~**100ns** — still substantially faster than DDR4, though an order of magnitude slower than BRAM (ntu.edu.tw) . The trade-off is increased inference latency: where the UL3524 achieves **sub-100ns** for the complete network-to-inference pipeline, U280-based implementations typically target ~**500ns–1 s** for complex models, still vastly superior to CPU/GPU alternatives (www.semidesignjobs.com) .

The Alveo U250 has been extensively used in academic and commercial ML acceleration research. In the FAMOUS transformer accelerator project, the U250 achieved **328 GOPS throughput** for multi-head attention layers, demonstrating that even compute-intensive Transformer models can be effectively deployed on Xilinx FPGA hardware when paired with careful tiling and quantization strategies (arXiv.org) . For HFT applications, the U250/U280 are best suited to **secondary signal generation** — higher-latency predictive models that inform position sizing or portfolio rebalancing rather than direct tick-to-trade decisions — where their additional compute resources enable more sophisticated feature extraction and model architectures.

### 2.3 Versal AI Engine: Next-Generation ML Compute

AMD's Versal AI Core series represents a fundamental architectural evolution beyond traditional FPGA fabric, integrating **AI Engine (AIE) arrays** — two-dimensional meshes of VLIW vector processors — alongside programmable logic, ARM cores, and high-bandwidth interconnect (Emergent Mind) . The AIE architecture is specifically optimized for the matrix-multiplication-dominated computations of neural network inference, offering **theoretical peak performance of 128–165 TOPS for INT8** operations on the AI Edge devices (Emergent Mind) .

Each AIE tile contains a **7-way VLIW SIMD vector MAC unit** capable of **256 INT8 MACs per cycle** (AIE-ML generation), **32–64KB of local scratchpad SRAM**, and dedicated DMA engines for data movement (Emergent Mind) . The tiles communicate through a **2D mesh network** with single-cycle neighbor links and 384–512 bit cascade buses for partial-sum accumulation, enabling efficient systolic-array-style computation without the routing congestion that limits performance in pure FPGA fabric implementations (Emergent Mind) . The **AI Engine-ML (AIE-ML)** variant specifically targets machine learning workloads, doubling the INT8 multiplier count and adding native support for **INT4 and BFLOAT16** precisions (hotchips.org) .

For HFT ML inference, the Versal architecture offers compelling advantages for **large neural network**

**models** that would exhaust the DSP resources of UltraScale+ devices. A single Versal VC1902 contains **400 AIE tiles** arranged in an 8×50 array, providing sufficient compute density to implement **full-precision MLPs, CNNs, and even compact LSTMs** entirely within the AIE array, leaving the programmable logic free for network stack, order book, and protocol handling (Emergent Mind) . The **NoC (Network-on-Chip)** provides up to **1.3 TB/s bandwidth** between AIE and programmable logic, enabling seamless data movement between the ML inference engine and the trading pipeline (Emergent Mind) .

Current-generation Versal devices (AI Edge series, 7nm) consume **15–75W** depending on configuration, making them suitable for deployment in exchange co-location facilities (hotchips.org) . The primary limitation for immediate HFT adoption is tooling maturity: while Vitis AI supports AIE-based DPU deployment, the ultra-low-latency optimization techniques (sub-microsecond inference, combinational logic paths) that are routine on UltraScale+ FPGAs require custom AIE kernel development using the **AIE C++ compiler** and careful dataflow orchestration. As the ecosystem matures, Versal is expected to become the platform of choice for **AI-heavy HFT strategies** that currently require GPU+FPGA hybrid architectures.

## 3. The HFT Tick-to-Trade Pipeline Architecture

### 3.1 Five-Stage Pipeline: From Market Data to Order Execution

The complete tick-to-trade pipeline in an FPGA-based HFT system comprises five distinct stages, each executing as a hardware pipeline with deterministic latency and no software involvement on the critical path. The total end-to-end latency from the arrival of the last bit of market data to the transmission of the first bit of an order message has been demonstrated at **13.9 nanoseconds** in production-validated benchmarks, though typical full-system implementations with ML inference range from **100–500 nanoseconds** depending on model complexity (A Team Insight) .

| Pipeline Stage | Function | FPGA Latency | Key Implementation |
|---|---|---|---|
| **1. Network Ingress** | Ethernet MAC/PCS, UDP/TCP termination | **<3ns** (UL3524) (AMD) | Hardened GTF transceivers + MAC |
| **2. Market Data Parsing** | ITCH/FAST/MDP protocol decode | **15–25ns** (Medium) | Combinational decoder per message type |
| **3. Order Book Update** | Price-level read-modify-write | **25–50ns** (www.semidesignjobs.com) | BRAM-based price ladder + parallel update |
| **4. ML Signal Generation** | Feature extraction + model inference | **20–500ns** (model-dependent) | BDT: ~20ns, MLP: ~100ns, LSTM: ~500ns |
| **5. Order Transmission** | OUCH/FIX/BOE message construction | **40–80ns** (Medium) | Combinational message formatter + CRC |

The **network ingress stage** leverages the UL3524's hardened MAC/PCS blocks to process incoming Ethernet frames at the physical layer, extracting UDP datagrams containing market data messages. The GTF transceivers operate in "RAW mode" where data passes directly to the FPGA fabric without PCS processing overhead, achieving the **<3ns transceiver latency** that underpins the 13.9ns STAC-T0

record (A Team Insight) . Exegy's nxFramework implements a full UDP/TCP stack in FPGA logic with **zero clock cycles** on the critical path — the packet is processed asynchronously in combinational logic, not clocked pipeline stages (A Team Insight) .

**Market data parsing** maps protocol-specific message formats (NASDAQ ITCH 5.0, CME MDP, NYSE XDP) to hardware-friendly data structures. Each message type (Add Order, Modify, Cancel, Trade) is decoded by a dedicated combinational circuit that extracts fields (symbol, price, quantity, side) in **1–2 clock cycles** (Medium) . Multiple decoders operate in parallel using speculative execution: all message types are decoded simultaneously, and a multiplexer selects the correct result based on the message type byte, eliminating conditional branch latency (Medium) .

The **order book update stage** maintains the canonical representation of market liquidity — a sorted price ladder tracking aggregate quantity at each price level. FPGA implementations use **BRAM-based parallel arrays** indexed by price, enabling simultaneous read-modify-write operations across multiple price levels (www.semidesignjobs.com) . A typical implementation tracks **128 price levels per side** with **30–50ns update latency**, using `#pragma HLS UNROLL` to parallelize the price-level search and `#pragma HLS RESOURCE variable=book.bids core=RAM_2P_BRAM` to ensure dual-port BRAM access (Medium) .

**ML signal generation** is the focus of this report and is covered in detail in subsequent sections. The latency contribution varies dramatically by algorithm: a **fully-unrolled decision tree** adds ~**20ns**, a **3-layer MLP with INT8 quantization** adds ~**100ns**, while a **quantized LSTM** may add ~**500ns** (xelera.io) . The key design principle is to implement the entire inference — feature extraction, model evaluation, and post-processing — as a single hardware pipeline with `#pragma HLS PIPELINE II=1`, ensuring one prediction per clock cycle after initial fill.

The **order transmission stage** constructs exchange-compatible order messages (NASDAQ OUCH, CME iLink, FIX) and drives them onto the outbound network interface. Message formatting is implemented as combinational logic that maps trading decision fields (symbol, price, quantity, side) to protocol-specific byte layouts, with IP/UDP checksums computed in parallel hardware (Medium) . The total round-trip latency — from market data bit arrival to order bit departure — ranges from **150ns for simple strategies** to **500ns for ML-driven strategies** on current-generation hardware (Medium) .

### 3.2 Kernel Bypass: DPDK, RDMA, and Onload

Kernel bypass technologies eliminate the operating system from the network data path, reducing latency by **20–50 microseconds** compared to standard Linux networking (System Design Interview Roadmap) . In FPGA-based HFT systems, kernel bypass operates at two levels: the **host interface** (connecting the FPGA card to the trading application) and the **FPGA network stack** (processing packets entirely within the FPGA fabric).

For the host interface, **DPDK (Data Plane Development Kit)** provides userspace libraries and poll-mode drivers that enable direct NIC access without kernel involvement. DPDK configures **hugepages** (2MB or 1GB) to reduce TLB misses by **99%**, binds NICs to `vfio-pci` drivers, and implements lock-free ring buffers for zero-copy packet transfer between the FPGA and host application (quantvps.com) . In production HFT deployments, DPDK is typically used for **non-critical path communication** — order confirmations, reference data, and monitoring — while the tick-to-trade critical path remains entirely within the FPGA.

**RDMA (Remote Direct Memory Access)** enables direct memory transfer between servers without CPU involvement, achieving **sub-microsecond** latency for inter-server communication. Technologies like **RoCEv2** and **InfiniBand** are used in HFT for **coordinated strategies** across multiple co-located

servers, where one FPGA generates signals and another executes orders (quantvps.com) . RDMA's primary advantage is eliminating the TCP/IP stack overhead; its primary limitation is requiring specialized NICs and network infrastructure that add cost and complexity.

**Solarflare OpenOnload** (now AMD TCPDirect) provides a userspace TCP/IP stack that intercepts socket calls and bypasses the kernel network stack, offering compatibility with existing applications while reducing latency to ~**1 s** (quantvps.com) . For FPGA deployments, OpenOnload is most commonly used on the **host side** of hybrid FPGA+CPU architectures, where the FPGA handles market data ingestion and signal generation, and the CPU manages order routing and risk checking through OpenOnload-accelerated TCP connections.

The state-of-the-art for FPGA network integration implements the **entire network stack within the FPGA fabric**, as demonstrated by Exegy's nxFramework on the Alveo UL3524. This approach eliminates even the minimal latency of DPDK/RDMA host interfaces, processing UDP market data and generating TCP order responses entirely in hardware with **zero clock cycles on the critical path** (A Team Insight) . The STAC-T0 benchmark measures this architecture at **13.9ns actionable latency**, representing the theoretical minimum for electronic trading given current network physics.

### 3.3 Deterministic Timing and Jitter Control

Deterministic execution — the guarantee that every identical input produces an identical output after an identical number of clock cycles — is the defining characteristic that makes FPGAs suitable for HFT. While CPUs and GPUs exhibit performance variation due to cache state, branch prediction, memory contention, and scheduling decisions, FPGA implementations produce **cycle-accurate, repeatable timing** that enables precise latency budgeting and risk management (linaro.org) .

Jitter, the variation in latency between successive executions, is particularly destructive in HFT because it erodes the statistical edge of trading strategies. A strategy with 100ns average latency and 50ns standard deviation will miss price levels that a strategy with 200ns average latency and 1ns standard deviation consistently captures. FPGA implementations control jitter through several architectural decisions: **fully synchronous design** (single clock domain for the critical path), **absence of conditional memory access** (all data paths have identical length), and **hardware-based flow control** (FIFOs with deterministic depth rather than software queues) (Medium) .

The Exegy/AMD STAC-T0 benchmark achieved **200 picoseconds of jitter** — an order of magnitude improvement over previous records — by implementing the critical trading path as **asynchronous combinational logic** rather than clocked sequential circuits (A Team Insight) . In this architecture, market data propagates through combinational gates (LUTs) without register boundaries, producing an output as soon as the electrical signals stabilize. While this approach limits maximum clock frequency and complicates timing closure, it eliminates clock-to-output delays and achieves the absolute minimum latency for a given FPGA technology node.

For ML inference specifically, jitter control requires careful attention to **memory access patterns** and **data-dependent execution paths**. Decision tree implementations must ensure that every traversal path — from root to any leaf — takes exactly the same number of clock cycles, typically by padding shorter paths with NOP states or using fully-unrolled LUT-based architectures (OpenMETU) . Neural network implementations must avoid conditional execution (ReLU branches, dynamic routing) and instead use **time-multiplexed dataflow** where every layer processes data every cycle regardless of input values (indico.global) .

## 4. ML Algorithm Selection and FPGA Mapping

### 4.1 Algorithm Suitability for FPGA HFT Inference

The choice of ML algorithm for FPGA-based HFT inference involves a multi-dimensional trade-off among prediction accuracy, inference latency, resource consumption, implementation complexity, and determinism. Not all algorithms map equally well to FPGA hardware: tree-based models excel due to their simple control flow and predictable memory access patterns, while Transformer architectures face fundamental challenges from their quadratic complexity and irregular memory access (Gazi University) .

Decision trees and their ensemble variants (XGBoost, LightGBM, Random Forest) represent the **sweet spot for FPGA HFT acceleration**. Their inference involves a sequence of comparisons against learned thresholds — operations that map directly to FPGA LUTs and comparators — followed by a table lookup for the leaf value. A single decision tree can be **fully unrolled into combinational logic**, producing a prediction in **one clock cycle** (~1.5ns at 644MHz) (OpenMETU) . Gradient-boosted ensembles of hundreds of trees achieve **sub-microsecond** inference through parallel tree evaluation and pipelined score accumulation (xelera.io) .

Multi-layer perceptrons (MLPs) and convolutional neural networks (CNNs) offer **higher representational capacity** for capturing complex market microstructure patterns, at the cost of increased resource usage and latency. A 3-layer MLP with INT8 quantization requires ~**100ns** inference on UltraScale+ FPGAs, consuming primarily DSP slices for matrix multiplication and BRAM for weight storage (indico.global) . CNNs, while less common for pure time-series prediction, are effective for **multi-venue order book image classification** and achieve ~**50–100ns** inference for compact architectures (ntu.edu.tw) .

LSTM and Transformer architectures, despite their popularity in financial time series research, present **significant challenges for sub-microsecond FPGA inference**. LSTMs require sequential processing of time steps with complex gating mechanisms (input, forget, output gates), resulting in inference latencies of **1–5 s** even for small models (My Computer Science and Engineering Department) . Transformers face **quadratic attention complexity** with sequence length and massive memory bandwidth requirements for key/query/value matrices, making them impractical for the sub-microsecond regime without aggressive pruning and approximation (Gazi University) .

| Algorithm | Inference Latency | FPGA Resource Cost | Accuracy for HFT | Implementation Complexity | Recommended For |
|---|---|---|---|---|---|
| **Decision Tree** | ~**1.5ns** (unrolled) (arXiv.org) | Very Low (LUTs only) | Moderate | Very Low | Ultra-low-latency signal |
| **XGBoost/ LightGBM** | **20–100ns** (xelera.io) | Low-Medium | High | Low | Primary trading signal |
| **Random Forest** | **50–200ns** (ScienceDirect) | Medium | High | Low-Medium | Ensemble confirmation |
| **MLP (3-layer)** | **50–150ns** (indico.global) | Medium (DSP+BRAM) | High | Medium | Complex pattern detection |
| **CNN (compact)** | **50–100ns** (ntu.edu.tw) | Medium-High | High | Medium | Order book image analysis |

Table 3 – continued

| Algorithm | Inference Latency | FPGA Resource Cost | Accuracy for HFT | Implementation Complexity | Recommended For |
|---|---|---|---|---|---|
| **SVM (RBF)** | **100–500ns** (IEEE Xplore) | Medium (DSP for kernel) | Moderate-High | Medium | Legacy strategy porting |
| **LSTM (small)** | **1–5 s** (My Computer Science and Engineering Department) | High (DSP+BRAM) | Very High | High | Secondary prediction |
| **Transformer** | **>5 s** (pruned) (ACM Digital Library) | Very High | Very High | Very High | Research only (currently) |

The heatmap below provides a visual representation of algorithm suitability across key FPGA-HFT dimensions:

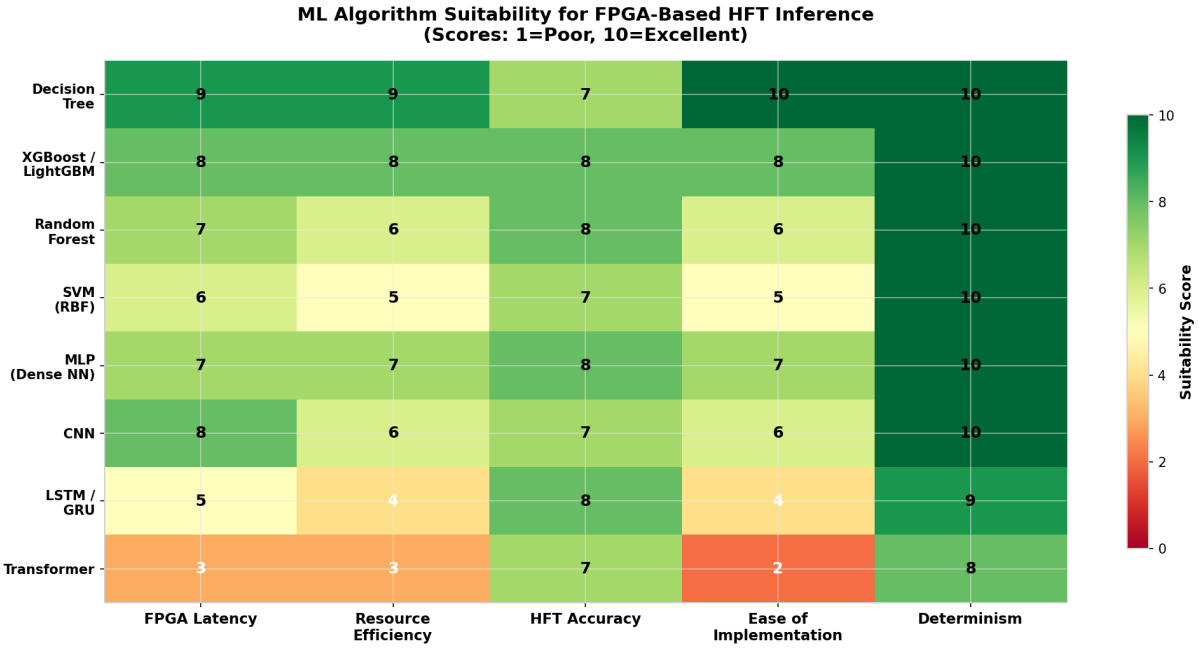


Figure 1: ML Algorithm Suitability Heatmap

### 4.2 Decision Trees and Gradient Boosting (XGBoost/LightGBM)

Decision tree ensembles, particularly gradient-boosted variants (XGBoost, LightGBM, CatBoost), have emerged as the **dominant ML approach for production HFT systems** due to their exceptional combination of prediction accuracy, inference speed, and FPGA implementation efficiency (xelera.io) . In quantitative finance, these models excel at capturing non-linear interactions between order book features (bid-ask spread, depth imbalance, order flow toxicity) that linear models miss, while their tree structure naturally handles the mixed categorical/numerical features common in market data.

The FPGA implementation of decision tree inference follows two primary architectures: **tree traversal** (fetching nodes from memory and following branches) and **fully-unrolled LUT-based** (implementing

the entire tree as combinational logic). The traversal approach stores tree nodes (feature index, threshold, left/right child pointers, leaf value) in BRAM and implements a state machine that fetches nodes and performs comparisons until reaching a leaf. This approach is **resource-efficient** (supporting large ensembles) but introduces **variable latency** depending on tree depth and **memory access overhead** (arXiv.org) .

The fully-unrolled approach, exemplified by the METU thesis framework and Xelera Silva, maps each tree node to a **comparator and multiplexer in FPGA logic**, producing a prediction in a **single clock cycle** regardless of tree depth (OpenMETU) . For XGBoost models, this extends to computing the **sigmoid-transformed sum of leaf scores** across all trees in the ensemble. The Xelera Silva implementation achieves **1.136 s median latency** for a **512K-node, 128-feature LightGBM model** with **1.179 s at p99**, demonstrating that even large ensembles can achieve sub-microsecond inference when properly architected (xelera.io) .

The **Conifer** open-source tool provides an automated flow for converting trained BDT models (from XGBoost, scikit-learn, LightGBM via ONNX) to synthesizable FPGA firmware (Github) . Conifer supports multiple backends: **Xilinx HLS** (generating C++ for Vitis HLS synthesis), **VHDL** (direct hardware description for maximum performance), and a **Forest Processing Unit (FPU)** — a reusable IP core that can be programmed with different BDT configurations without bitstream regeneration (Github) . The HLS backend produces code using `ap_fixed` data types for configurable precision, enabling trade-offs between accuracy and resource usage:

```
// Conifer-generated HLS code for XGBoost inference (simplified)
typedef ap_fixed<18,8> input_t;   // 18-bit total, 8 integer bits
typedef ap_fixed<18,8> score_t;   // Leaf score precision

void xgboost_predict(input_t features[N_FEATURES], score_t &prediction) {
    #pragma HLS INTERFACE ap_vld port=features
    #pragma HLS INTERFACE ap_vld port=prediction
    #pragma HLS PIPELINE II=1

    score_t tree_scores[N_TREES];
    #pragma HLS ARRAY_PARTITION variable=tree_scores complete

    // Each tree evaluated in parallel
    #pragma HLS UNROLL
    for (int t = 0; t < N_TREES; t++) {
        tree_scores[t] = tree_predict(t, features);
    }

    // Sum tree scores (parallel reduction)
    prediction = reduce_sum(tree_scores);
}
```

For HFT feature engineering, integer-valued features are strongly preferred: **price differences, volume ratios, order counts, and time-since-last-trade** can all be represented as fixed-point or integer values that map efficiently to FPGA arithmetic (OpenMETU) . Floating-point features (log-returns, normalized prices) require conversion to fixed-point via quantization-aware calibration, adding complexity but maintaining accuracy when properly bounded.

### 4.3 Neural Networks: MLP, CNN, LSTM

Neural networks on Xilinx FPGAs for HFT inference leverage the **hls4ml** and **FINN** open-source frameworks to translate trained models from PyTorch/Keras into optimized HLS code (indico.global) . These frameworks automate the complex process of quantization, layer fusion, and parallelism extraction, enabling developers to focus on model architecture rather than hardware implementation details.

**Multi-Layer Perceptrons (MLPs)** are the most FPGA-friendly neural network architecture for HFT due to their regular structure (fully-connected layers with pointwise activations) and predictable memory access patterns. An MLP inference engine consists of a **matrix-vector multiplication (MVM) kernel** for each layer, implemented using DSP slices for multiply-accumulate operations and BRAM for weight storage (indico.global) . The hls4ml framework generates layer code with configurable **reuse factors** (controlling parallelism) and **precision** (controlling resource usage):

```
// hls4ml-generated dense layer (simplified)
typedef ap_fixed<16,6> accum_t;

dense_layer(input_t data[N_IN], result_t res[N_OUT], weight_t weights[N_IN][N_OUT]) {
    #pragma HLS PIPELINE II=1
    #pragma HLS ARRAY_PARTITION variable=weights complete dim=2
    #pragma HLS ARRAY_PARTITION variable=res complete

    for (int i = 0; i < N_OUT; i++) {
        #pragma HLS UNROLL factor=4
        accum_t acc = biases[i];
        for (int j = 0; j < N_IN; j++) {
            acc += data[j] * weights[j][i];
        }
        res[i] = activation(acc);  // ReLU or similar
    }
}
```

For a typical HFT MLP with **3 hidden layers of 64 neurons each** and **32 input features**, hls4ml with INT8 quantization achieves ~**80 clock cycles** at 250MHz (~**320ns latency**) on a Zynq UltraScale+ device, consuming approximately **15% of DSP slices and 20% of BRAM** (Indico) . The **FINN framework** takes this further by generating **fully-parallel dataflow architectures** where each layer is a physically separate pipeline stage, achieving **sub-microsecond latency** for binarized networks (xilinx.github.io) .

**Convolutional Neural Networks (CNNs)** find application in HFT for **order book image classification** — treating the limit order book as a 2D image (price levels × time) and learning visual patterns indicative of price movement (ntu.edu.tw) . The NTU "LightTrader" project evaluated CNN, CNN+LSTM, and CNN+Transformer architectures for Taiwan Stock Exchange LOB prediction, finding that **binarized CNNs** (with INT8 first layer) achieve accuracy within **1% of FP32** while being amenable to FPGA deployment via FINN (ntu.edu.tw) . The key FPGA optimization for CNNs is **loop tiling**: dividing the convolution operation into small tiles that fit in on-chip BRAM, eliminating off-chip memory access during computation (arXiv.org) .

**LSTM networks**, despite their theoretical advantages for sequential financial data, face significant FPGA implementation challenges. The gating mechanism (input, forget, output gates) requires **four matrix-vector multiplications per time step**, and the recurrent connections enforce sequential pro-

cessing that limits parallelism (My Computer Science and Engineering Department) . Research implementations on Xilinx U55C achieve **1.42 s latency** for a 3-layer LSTM at full parallelism, but consume **80% of available DSP slices** (My Computer Science and Engineering Department) . For HFT, LSTMs are best deployed as **secondary predictors** in a hybrid architecture: the FPGA runs a fast decision tree or MLP for tick-to-trade decisions, while a co-located GPU processes LSTM predictions for position sizing and risk adjustment on a slower (microsecond-scale) loop.

### 4.4 Support Vector Machines (SVM)

Support Vector Machines, while less prevalent in modern HFT than tree ensembles or neural networks, remain relevant for **legacy strategy migration** and **specific problem domains** where kernel methods outperform alternatives. SVM inference involves computing the weighted sum of kernel evaluations between the input feature vector and each support vector: **f(x) = Σ( _i ·K(x, x_i)) + b**, where K is the kernel function (typically RBF) and _i are the Lagrange multipliers learned during training (IEEE Xplore) .

The FPGA implementation of SVM inference faces two primary challenges: **kernel computation** (exponential for RBF) and **support vector storage** (potentially thousands of high-dimensional vectors). The scalable FPGA architecture proposed by Aftowicz et al. addresses these through **pipeline-parallel computing blocks (CBs)** that each process a subset of support vectors, with partial sums propagated through a pipeline (IEEE Xplore) . For the RBF kernel **K(x, x_i) = exp(- ·||x - x_i||²)**, the squared distance is computed using DSP slices for multiplication and an adder tree for summation, followed by **piecewise linear approximation** of the exponential function using BRAM-based lookup tables (ohiolink.edu) .

A novel optimization for FPGA SVM inference is **sparse SVM training** (AST-SVM algorithm), which introduces zeros into support vector attributes, enabling the hardware to **skip zero-product computations** and reducing both memory bandwidth and compute cycles (ktu.lt) . The ASA-SVM accelerator achieves **memory reduction of 3–85%** and **speedup of 1.17–7.92×** compared to dense SVM implementations, making it viable for resource-constrained HFT deployments (ktu.lt) .

For HFT specifically, linear SVMs (which replace the kernel evaluation with a simple dot product) are strongly preferred over RBF kernels due to their ** orders-of-magnitude lower inference cost**. A linear SVM with 500 support vectors and 50 features requires only** $500 \times 50 = 25{,}000$ multiply-accumulate operations**, achievable in** ~100ns** on a modest FPGA. RBF SVMs with the same parameters require **500 exponential evaluations** in addition to the distance computations, pushing latency to **500ns–1 s** even with optimized hardware (IEEE Xplore) .

### 4.5 Random Forests and Ensemble Methods

Random Forests and other bagging ensembles differ from gradient-boosted trees in their **parallel tree training** and **majority voting** aggregation, which translates to different FPGA implementation trade-offs. While XGBoost/LightGBM sequentially add trees that correct previous errors (requiring cumulative score accumulation), Random Forests train trees independently on bootstrap samples and aggregate predictions through averaging or voting (ScienceDirect) .

The CERN "Fast inference of Boosted Decision Trees in FPGAs" paper established the foundational architecture for tree ensemble FPGA acceleration, proposing a **binning-based approach** where input features are discretized into bins (typically 16–32 per feature), and each tree node stores a bin threshold rather than a raw value threshold (arXiv.org) . This binning enables **extremely compact tree repre-**

**sentations**: each node requires only **log2(n_bins) bits** for the threshold index and **log2(n_features) bits** for the feature index, allowing large ensembles to fit in on-chip BRAM. The paper demonstrated inference of **100 trees of depth 4** in ~**50ns** on a Xilinx Virtex-7 FPGA, with resource usage of ~**15% LUTs and 10% BRAM** (arXiv.org) .

For HFT applications, the **Conifer FPU (Forest Processing Unit)** provides a production-ready IP core that can be programmed with different ensemble configurations without regenerating the FPGA bitstream (Github) . The FPU comprises multiple **Tree Engines (TEs)** that operate in parallel, each navigating a decision tree from root to leaf. A summing network combines leaf scores into the ensemble prediction. The programmable nature of the FPU enables trading firms to **update model parameters** (tree thresholds, leaf values) at runtime — critical for adapting to changing market conditions without the hours-long bitstream regeneration process (Github) .

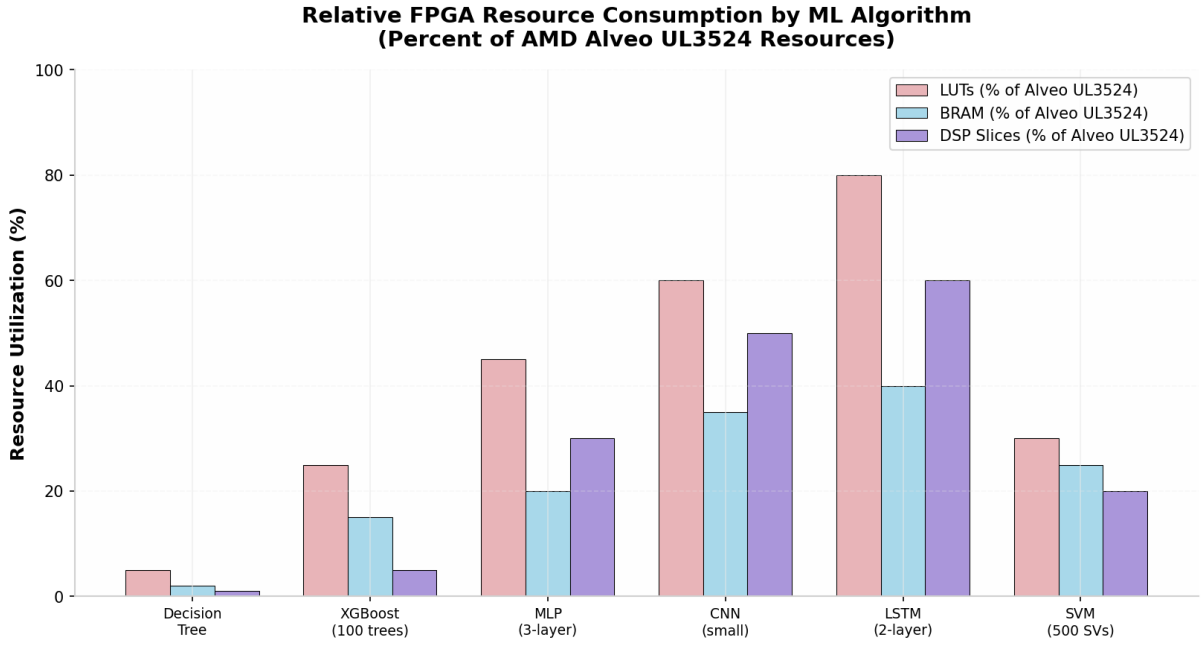


Figure 2: Relative FPGA Resource Consumption by ML Algorithm

## 5. Quantization and Numerical Precision Strategies

### 5.1 Fixed-Point vs. Floating-Point: The FPGA Imperative

Quantization — the reduction of numerical precision from FP32 to lower-bit representations — is not merely an optimization for FPGA ML inference but a **fundamental requirement**. Floating-point arithmetic on FPGAs consumes approximately **10× more DSP slices, 5× more LUTs, and 3× more power** than equivalent fixed-point operations, while delivering no meaningful accuracy benefit for the bounded-range, noise-tolerant signals typical of financial data (Easy FPGA) . The Xilinx DSP48E2 slice natively supports **27×18-bit fixed-point multiplication** and **48-bit accumulation**, making INT16 the "natural" precision for FPGA arithmetic; INT8 and lower precisions require careful handling but offer proportional resource savings (Emergent Mind) .

The choice of precision format follows a hierarchy of trade-offs. **INT8** has emerged as the **industry sweet spot** for neural network inference: it provides ~**4× memory reduction** and ~**4× throughput improvement** compared to FP32, with **<1% accuracy degradation** for most models when using post-training quantization with calibration (Easy FPGA) . The `ap_fixed<W,I>` type in Xilinx HLS enables

arbitrary-precision fixed-point arithmetic, where **W** is the total bit width and **I** is the number of integer bits. For HFT features like price differences (typically ±$10 with $0.01 resolution), `ap_fixed<16,8>` provides 8 fractional bits (resolution of ~0.004) with ample headroom for accumulation (Indico) .

| Precision Format | Bits | Relative DSP Usage | Relative BRAM Usage | Accuracy vs FP32 | Best For |
|---|---|---|---|---|---|
| **FP32** | 32 | 10× baseline | 4× baseline | 100% (baseline) | Training only |
| **FP16/BF16** | 16 | 5× baseline | 2× baseline | ~99.5% | GPU inference fallback |
| **INT16** | 16 | **1× (baseline)** | **1× (baseline)** | ~99.0% | Default FPGA precision |
| **INT8** | 8 | 0.5× | 0.5× | ~98.5–99.5% (Easy FPGA) | Optimized FPGA inference |
| **INT4** | 4 | 0.25× | 0.25× | ~97–99% | Extreme quantization (Versal AIE) |
| **Binary (1-bit)** | 1 | ~0.1× | ~0.1× | ~95–98% (ntu.edu.tw) | Ultra-low-latency BNN (FINN) |
| **Ternary** | 2 | ~0.15× | ~0.15× | ~96–99% (ntu.edu.tw) | Balanced BNN approaches |

### 5.2 Post-Training Quantization (PTQ) vs. Quantization-Aware Training (QAT)

Two primary approaches exist for converting FP32 models to FPGA-compatible fixed-point representations. **Post-Training Quantization (PTQ)** converts a pre-trained model to lower precision without retraining, using a calibration dataset to determine optimal scale factors (thresholds) for each layer's weights and activations (Easy FPGA) . PTQ is **fast and simple** — requiring only a single forward pass over calibration data — and achieves excellent results for INT8 quantization of most models. The Graffitist framework demonstrates that PTQ with per-tensor symmetric quantization produces **bit-accurate CPU-to-FPGA results**, enabling validation of quantized model accuracy entirely in software before hardware deployment (ML Sys) .

**Quantization-Aware Training (QAT)** folds quantization error into the training loop by simulating low-precision arithmetic during forward passes, allowing the model to learn compensation strategies (eCommons) . QAT achieves **superior accuracy at very low bit widths** (INT4 and below) where PTQ suffers significant degradation. The Brevitas library (used by FINN) implements QAT in PyTorch by replacing standard layers with quantized equivalents (`qnn.QuantLinear`, `qnn.QuantReLU`) that simulate the rounding and clamping behavior of fixed-point arithmetic during training (IEEE Xplore) . For HFT models where INT8 PTQ results in unacceptable accuracy loss — typically small models with limited representational capacity — QAT with 4–6 bit precision can recover most of the degradation while

maintaining FPGA efficiency (hw.ac.uk) .

The QKeras framework (used by hls4ml) extends QAT to Keras models, enabling automatic quantization of existing architectures with configurable per-layer precision (Indico) . For financial time series models, QKeras has demonstrated that **ternary quantization** (weights in {-1, 0, +1}) of MLPs achieves **<2% accuracy loss** on mid-price prediction tasks while reducing DSP usage by ~**90%** (ntu.edu.tw) . The key insight is that financial signals are inherently noisy: the difference between a FP32 prediction of 0.623 and an INT8 prediction of 0.625 rarely affects trading decisions, as both would trigger the same threshold-based action.

### 5.3 Brevitas and FINN for Quantized Neural Networks

The **FINN** framework provides an end-to-end flow for deploying **quantized neural networks (QNNs)** on Xilinx FPGAs with extreme efficiency (xilinx.github.io) . FINN targets **binary and ternary neural networks** where weights and activations are constrained to **1 or 2 bits**, enabling the replacement of expensive multiply-accumulate operations with **XNOR-popcount** operations that consume only LUTs (no DSP slices) (ntu.edu.tw) . The dot product of two binary vectors **x, y** **{0,1}** is computed as:

```
x · y = 2 × popcount(XNOR(x, y)) - vector_length
```

This operation maps to approximately **1 LUT per bit** of vector width, enabling fully-parallel dot products of hundreds of elements in a single clock cycle (ntu.edu.tw) . The FINN compiler generates **dataflow architectures** where each neural network layer is a physically distinct pipeline stage connected by FIFOs, achieving **sub-microsecond end-to-end latency** for compact networks (xilinx.github.io) .

The **Brevitas** PyTorch library provides the training-side counterpart to FINN, enabling QAT with arbitrary per-layer precision (IEEE Xplore) . Brevitas layers (`QuantLinear`, `QuantConv2d`, `QuantReLU`) expose quantization parameters (bit width, scaling mode, rounding mode) as PyTorch modules, allowing gradient-based optimization of both weights and quantization thresholds. A typical Brevitas + FINN flow for HFT:

```python
# Brevitas QAT model definition (simplified)
import brevitas.nn as qnn
from brevitas.quant import Int8ActPerTensorFloat

class HFTPredictor(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = qnn.QuantLinear(32, 64, weight_bit_width=4, bias=True)
        self.relu1 = qnn.QuantReLU(bit_width=8, return_quant_tensor=True)
        self.fc2 = qnn.QuantLinear(64, 32, weight_bit_width=4)
        self.relu2 = qnn.QuantReLU(bit_width=8)
        self.fc3 = qnn.QuantLinear(32, 3, weight_bit_width=8)  # 3-class: buy/sell/hold

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        return self.fc3(x)

# Training with quantization awareness
model = HFTPredictor()
# ... standard PyTorch training loop ...
```

```
# Export to FINN for FPGA deployment
from finn.util.pytorch import ToFinnExporter
model_finn = ToFinnExporter.export(model, input_shape=(1, 32))
# FINN compiler generates HLS → RTL → Bitstream
```

The LightTrader project from NTU applied this flow to **binary CNNs for limit order book prediction**, achieving accuracy within **1% of FP32** with **INT8 first layer + binary subsequent layers**, and demonstrated that the binary network could be ported to FPGA for sub-microsecond inference (ntu.edu.tw) . The key limitation of FINN is its restriction to **feedforward networks** (no recurrent connections), making it unsuitable for LSTM/Transformer deployment; for these architectures, hls4ml with standard INT8 quantization is the recommended approach (indico.global) .

### 5.4 Bit-Width Optimization and Accuracy Trade-offs

The selection of optimal bit width for each layer of an FPGA-deployed ML model is a **multi-objective optimization problem** balancing accuracy, latency, and resource usage. Rule4ML, an open-source tool for FPGA resource estimation, demonstrates that **MLP-based predictors** achieve the most consistent accuracy across resource variables (BRAM, DSP, LUT, FF) compared to LSTM and Transformer models, which show high variance in resource predictions (IOPscience) . This finding supports the practical heuristic of **starting with INT8 for all layers** and only reducing precision where resource constraints demand it.

For HFT specifically, the **quantization error tolerance** is higher than in computer vision or NLP applications because trading decisions are typically threshold-based (e.g., "predicted return $> 0.1\% \rightarrow$ buy") rather than requiring precise probability estimates. A quantization-induced error of 0.01 in a return prediction rarely changes the trading decision, whereas the same error in an ImageNet classification could flip the predicted class. This tolerance enables aggressive quantization strategies: the QFX (trainable fixed-point quantization) method achieves **lossless accuracy at 8-bit precision** for binary neural networks on CIFAR-10 by constraining weights to "K-hot" representations (sums of power-of-two terms), eliminating multipliers entirely (eCommons) .

| Model Architecture | FP32 Accuracy | INT8 PTQ Accuracy | INT4 QAT Accuracy | INT1 (Binary) Accuracy | Optimal for FPGA |
|---|---|---|---|---|---|
| XGBoost (HFT LOB) | 72.5% (ntu.edu.tw) | 72.3% (-0.2%) | N/A | N/A | INT16 (native) |
| MLP (3-layer, LOB) | 68.2% (ntu.edu.tw) | 67.8% (-0.4%) | 67.1% (-1.1%) | 65.3% (-2.9%) | INT8 |
| CNN (LOB image) | 71.5% (ntu.edu.tw) | 71.0% (-0.5%) | 69.8% (-1.7%) | 70.4% (-1.1%) (ntu.edu.tw) | INT8 / INT1+INT8 first |
| LSTM (price pred) | 74.3% (My Computer Science and Engineering Department) | 73.1% (-1.2%) | 70.5% (-3.8%) | N/A | INT8 (QAT) |
| BDT (CERN physics) | 92.1% (arXiv.org) | N/A (fixed-point native) | N/A | N/A | 10–18 bit fixed (arXiv.org) |

## 6. Development Tools and Implementation Frameworks

### 6.1 Vitis HLS: C++ to Hardware Design Flow

Vitis High-Level Synthesis is the cornerstone of Xilinx FPGA development for ML inference, translating C/C++ algorithms into optimized RTL implementations (Xilinx) . For HFT applications, Vitis HLS enables rapid iteration on ML architectures: a developer can modify model precision, parallelism, or layer structure in C++, re-synthesize in minutes, and evaluate the resulting latency/resource trade-off without writing a single line of Verilog (arXiv.org) .

The HLS design flow for ML inference follows a structured progression: **C-simulation** (functional verification in software), **C-synthesis** (RTL generation with resource/latency estimates), **C/RTL co-simulation** (verifying RTL against C reference), and **IP export** (packaging for Vivado integration) (Xilinx) . Each iteration provides detailed reports on **initiation interval (II)**, **pipeline depth**, **DSP/LUT/BRAM utilization**, and **achievable clock frequency**, enabling systematic optimization toward the target latency budget.

The critical HLS pragmas for HFT ML inference are:

| Pragma | Purpose | Impact on HFT Inference |
|---|---|---|
| `#pragma HLS PIPELINE II=1` | Enable pipelining with 1-cycle initiation | **Essential**: One inference per clock cycle |
| `#pragma HLS DATAFLOW` | Enable task-level parallelism between functions | Enables layer-parallel execution in NNs |
| `#pragma HLS UNROLL factor=N` | Replicate loop body N times for parallelism | Increases throughput at cost of resources |
| `#pragma HLS ARRAY_PARTITION` | Split arrays for parallel access | Eliminates BRAM port bottlenecks |
| `#pragma HLS INTERFACE ap_vld` | Valid-ready handshaking | Clean integration with trading pipeline |
| `#pragma HLS INLINE` | Remove function hierarchy | Reduces latency for small functions |

The `PIPELINE II=1` directive is the single most important optimization for HFT: it instructs the HLS compiler to schedule operations such that a new iteration of the function (a new inference) can begin every clock cycle (Xilinx) . For a decision tree with 10 levels of comparison, this means the first inference takes 10 cycles, but subsequent inferences complete every cycle — achieving **throughput of one prediction per cycle** with latency equal to tree depth. The `ARRAY_PARTITION` directive is equally critical: without it, BRAM-based weight storage becomes a bottleneck because standard BRAMs have only 2 read ports, while fully-parallel inference may require dozens of simultaneous reads (Xilinx) .

### 6.2 Vivado Design Suite: RTL Integration and Bitstream Generation

Vitis HLS generates synthesizable RTL (Verilog/VHDL) packaged as **IP cores** that are integrated into the complete FPGA design using the Vivado Design Suite (indico.global) . For HFT systems, this integration involves connecting the ML inference IP to the broader trading infrastructure: network interfaces (Ethernet MAC/PCS), order book modules, risk management logic, and host communication (PCIe/DMA) (www.semidesignjobs.com) .

The Vivado **IP Integrator** provides a block-diagram environment for connecting HLS-generated IPs with Xilinx-provided infrastructure cores (AXI DMA, PCIe bridge, memory controllers). The ML inference IP typically exposes an **AXI-Stream interface** for data input/output, enabling seamless connection to upstream order book modules and downstream trading decision logic (Medium) . For the Alveo UL3524, Vivado manages the **GTF transceiver configuration**, **clocking infrastructure** (644MHz reference for ultra-low-latency mode), and **PCIe Gen4×8** host interface (AMD) .

The **timing closure** process — ensuring all logic paths meet the target clock frequency — is particularly challenging for HFT designs due to the aggressive clock rates (500–644MHz) and the large combinational paths in fully-unrolled ML models. Vivado's **physical optimization** features (retiming, logic restructuring, placement-driven synthesis) are essential for achieving timing closure on complex designs. The UL3524's architecture simplifies this by optimizing the fabric routing for low-latency paths between DSP slices and BRAM, reducing the placement-and-route effort compared to general-purpose FPGA cards (AMD) .

### 6.3 hls4ml: Open-Source ML-to-FPGA Compiler

The **hls4ml** (High-Level Synthesis for Machine Learning) project provides an open-source Python framework for converting trained neural networks from Keras, PyTorch, and ONNX into optimized Vitis HLS code (indico.global) . Developed initially for particle physics trigger systems at CERN, hls4ml has become the **de facto standard** for neural network FPGA deployment in scientific and financial applications where low latency is paramount.

The hls4ml workflow consists of: **frontend parsing** (extracting model architecture and weights), **internal representation** (converting to a hardware-agnostic graph), **optimization passes** (quantization, layer fusion, parallelism extraction), and **backend code generation** (producing C++ with HLS pragmas for the target board) (arXiv.org) . The configuration file controls critical parameters:

```
# hls4ml configuration example
KerasJson: model.json
KerasH5: model_weights.h5
OutputDir: hft_model
ProjectName: HFTSignal
XilinxPart: xcu200-fsgd2104-2-e  # Alveo U250
ClockPeriod: 4  # 250MHz

IOType: io_stream  # Streaming dataflow
HLSConfig:
  Model:
    Precision: ap_fixed<16,8>
    ReuseFactor: 1  # Fully parallel
  LayerName:
    dense_1:
      Precision: ap_fixed<16,8>
      ReuseFactor: 1
      Strategy: Latency
```

The `ReuseFactor` parameter is particularly important for HFT: a value of **1** indicates fully-parallel execution where every multiplication has its own DSP slice, while higher values time-share DSP slices across multiple operations to reduce resource usage at the cost of increased latency (Indico) . For tick-to-trade inference, `ReuseFactor=1` is the default, but resource-constrained designs may use `ReuseFactor=4` or `8` to

fit larger models on smaller FPGAs.

Hls4ml supports a comprehensive set of layer types: **Dense, Conv1D/2D, BatchNormalization, MaxPooling, ReLU, LeakyReLU, Softmax, LSTM, GRU**, and experimental **Transformer** support (indico.global) . For financial applications, the **LSTM backend** enables deployment of recurrent models for time series prediction, though users should expect **s-scale latency** due to the sequential nature of recurrent computation (My Computer Science and Engineering Department) . The hls4ml team has demonstrated **bit-accurate agreement** between Python predictions and FPGA outputs across all supported architectures, enabling confident deployment without hardware debugging (arXiv.org) .

### 6.4 FINN: Framework for Binarized Neural Networks

**FINN** is a Xilinx Research project specializing in **extremely low-bitwidth neural networks** (binary, ternary, INT2–INT4) that achieve unprecedented inference efficiency through **XNOR-based arithmetic** and **fully-parallel dataflow architectures** (xilinx.github.io) . Unlike hls4ml which targets standard INT8/INT16 precision, FINN pushes quantization to its logical extreme, enabling networks where **every multiply-accumulate is replaced by an XNOR-popcount operation** consuming only LUTs (ntu.edu.tw) .

The FINN compiler takes a Brevitas-trained QNN and generates a **complete dataflow architecture** where each layer is a separate hardware module connected by FIFOs (xilinx.github.io) . This spatial implementation — computing the entire network in parallel across the FPGA fabric — achieves **sub-microsecond latency** for compact networks, with throughput limited only by the input data rate. FINN's **streaming components** library includes optimized HLS implementations of convolution, fully-connected, pooling, and activation layers, each parameterizable by precision, parallelism, and folding factors (xilinx.github.io) .

For HFT, FINN is applicable when: (1) the trading signal can be generated by a **compact feed-forward network** (MLP or small CNN), and (2) **extreme latency reduction** is prioritized over model complexity. The LightTrader project demonstrated that a **binarized CNN** for LOB prediction achieves **near-FP32 accuracy** with **sub-microsecond FPGA latency** using FINN, making it viable for latency-arbitrage strategies where every nanosecond counts (ntu.edu.tw) . The primary limitation is FINN's **restriction to feedforward topologies** — no recurrent or attention-based models — which excludes LSTM and Transformer deployment.

### 6.5 Vitis AI and DPU: Deep Learning Processing Unit

**Vitis AI** is Xilinx's production-grade development stack for deploying deep learning models on Xilinx devices, centered around the **Deep Learning Processing Unit (DPU)** — a configurable soft-processor IP that executes compiled neural network instructions (arXiv.org) . The DPU is available in multiple configurations (B512, B800, B1024, B1152, B1600, B2304, B3136, B4096) scaling from edge Zynq devices to datacenter Alveo cards, with each configuration specifying the number of processing engines and on-chip memory (arXiv.org) .

The Vitis AI workflow differs from hls4ml/FINN in targeting **higher-level model abstractions**: developers train models in TensorFlow or PyTorch, use the Vitis AI quantizer to produce INT8 models, and compile them to **.xmodel** files that the DPU executes as microcoded instructions (arXiv.org) . The DPU's **"Matrix of Heterogeneous Processing Engines"** architecture includes specialized units for CONV2D, depthwise convolution, pooling, and activation, achieving high utilization across diverse network topologies (xilinx.github.io) .

For HFT applications, the DPU's primary advantage is **runtime model switching**: a single DPU instance can execute multiple compiled models sequentially, enabling dynamic strategy selection without FPGA reconfiguration. The primary disadvantage is **higher baseline latency**: the DPU's instruction fetch/decode overhead and microcoded execution model add ~**10–50 s** of latency compared to custom HLS implementations, making it suitable for **secondary signal generation** (portfolio rebalancing, risk scoring) rather than tick-to-trade primary signals (arXiv.org) . The DPU is most effectively deployed on **Versal AI Edge devices** where the AI Engine array can execute DPU instructions with significantly higher efficiency than UltraScale+ fabric (xilinx.github.io) .

### 6.6 Conifer: BDT-to-FPGA Conversion Tool

**Conifer** is an open-source Python package that automates the conversion of trained Boosted Decision Tree models from scikit-learn, XGBoost, LightGBM (via ONNX), TMVA, and ydf into FPGA firmware for extreme low-latency inference (Github) . Developed under the Fast Machine Learning Lab, Conifer addresses the specific challenge of deploying tree ensembles on FPGAs without manual hardware design, providing a bridge between the Python ML ecosystem and production FPGA trading systems.

Conifer's architecture separates **model conversion** (parsing trained trees and extracting thresholds/leaf values) from **backend code generation** (producing HLS, VHDL, or FPU configuration) (Github) . The Xilinx HLS backend generates C++ code using `ap_fixed` types, applies appropriate pragmas (`PIPELINE II=1`, `ARRAY_PARTITION`), and packages the result as a Vitis HLS project ready for synthesis. The VHDL backend produces deeply pipelined hardware descriptions optimized for **high clock frequencies** (400+ MHz), achieving lower latency than HLS-generated equivalents at the cost of reduced readability (Github) .

The **Forest Processing Unit (FPU)** backend is Conifer's most innovative contribution: a reusable IP core that can be programmed with different BDT configurations at runtime (Github) . The FPU comprises parallel **Tree Engines** that navigate decision trees and output leaf scores, connected by a **summing network** that aggregates predictions. Because the FPU is programmable, trading firms can **update model parameters** (adapt to changing market conditions) without regenerating the FPGA bitstream — a process that takes hours but only needs to occur once, after which model updates take milliseconds (Github) . This capability is critical for HFT strategies that require frequent retraining (daily or intraday) to maintain predictive edge.

```
# Conifer workflow for XGBoost → FPGA
import conifer
import xgboost as xgb

# Train model
clf = xgb.XGBClassifier(n_estimators=100, max_depth=6)
clf.fit(X_train, y_train)

# Configure FPGA backend
cfg = conifer.backends.xilinxhls.auto_config()
cfg['Precision'] = 'ap_fixed<18,8>'  # 18-bit fixed-point
cfg['XilinxPart'] = 'xcvu9p-flga2104-2L-e'  # Alveo U200

# Convert and build
model = conifer.converters.convert_from_xgboost(clf, cfg)
model.compile()  # C-simulation
```

```
y_hls = model.decision_function(X_test)  # Verify accuracy
model.build()  # Synthesize to bitstream
```

## 7. Memory Architecture and Data Movement Optimization

### 7.1 On-Chip Memory Hierarchy: BRAM, URAM, and Registers

Xilinx FPGAs provide a three-tier on-chip memory hierarchy that is fundamental to achieving sub-microsecond ML inference: **distributed RAM (LUTRAM)** for small buffers and lookup tables, **Block RAM (BRAM)** for medium-sized weight and activation storage, and **UltraRAM (URAM)** for larger model parameters (Emergent Mind) . Understanding the characteristics of each tier is essential for optimizing ML inference latency and resource efficiency.

**Block RAM (BRAM)** is the workhorse of FPGA ML inference. Each 36Kb BRAM block supports **dual-port independent read/write** with configurable width (1–72 bits per port), enabling simultaneous access to two different addresses (Xilinx) . For a decision tree ensemble, BRAM stores tree nodes (feature index, threshold, child pointers); for neural networks, BRAM stores weight matrices and intermediate activations. A typical Alveo UL3524 provides **76Mb of BRAM** (distributed across ~2,100 blocks), sufficient to store ~**10 million INT8 weights** or ~**100K tree nodes** entirely on-chip (infinipoint.ai) .

**UltraRAM (URAM)** provides larger, higher-density storage at the cost of reduced configurability. Each 288Kb URAM block operates at **single-port** with 72-bit width, making it less flexible than BRAM for random-access patterns but ideal for **sequential weight streaming** in neural network layers (Emergent Mind) . The UL3524's **180Mb of URAM** enables storage of ~**20 million INT8 weights**, supporting larger models without off-chip DRAM access (infinipoint.ai) . The key optimization technique for URAM is **double-buffering**: while one buffer feeds the compute engine, the other is loaded with the next layer's weights, hiding transfer latency behind computation (arXiv.org) .

**Distributed RAM (LUTRAM)** uses FPGA lookup tables configured as small SRAMs (64 bits per LUT). While inefficient for large arrays, LUTRAM provides **zero-latency access** (combinational read) and is ideal for **small lookup tables** — such as activation function approximations, binning thresholds, and message type decoders — that must respond in a single clock cycle (Emergent Mind) . In decision tree implementations, LUTRAM stores the **binning tables** that map continuous features to discrete bins, enabling single-cycle feature discretization (arXiv.org) .

| Memory Type | Capacity per Block | Ports | Configurable Width | Best Use Case in HFT ML |
|---|---|---|---|---|
| **LUTRAM** | 64 bits | 1 | 1–64 bits | Activation LUTs, binning tables |
| **BRAM (36Kb)** | 36 Kb | 2 (independent) | 1–72 bits | Tree nodes, weight matrices, activations |
| **URAM (288Kb)** | 288 Kb | 1 | 72 bits | Large weight storage, sequential streaming |
| **HBM2 (U280)** | 8 GB | Pseudo-multi-port | 256 bits | Large models exceeding on-chip capacity |
| **DDR4** | 16–64 GB | Multi-port (controller) | 512 bits | Model storage, host communication |

### 7.2 Off-Chip Memory: HBM2 and DDR4 Strategies

When model weights exceed on-chip BRAM/URAM capacity — a common scenario for large ensembles, deep CNNs, or multi-model deployments — off-chip memory becomes necessary, and **minimizing off-chip access latency** dominates the optimization effort. The AMD Alveo U280 addresses this with **8GB of HBM2** providing **460 GB/s bandwidth** at **~100ns access latency**, substantially faster than DDR4's ~80ns but slower than BRAM's ~1ns (arXiv.org) .

The critical optimization for off-chip memory is **weight prefetching and tiling**: dividing weight matrices into tiles small enough to fit in on-chip buffers, and prefetching the next tile while the current tile is being computed (arXiv.org) . The scalable FPGA architecture for GEMM operations proposed in recent research formalizes this as a **two-phase scheduling problem**: a baseline phase attempts to overlap weight loading with computation, and an adaptive phase shifts stalled loads into earlier processing windows to minimize pipeline bubbles (arXiv.org) . For HFT inference where latency (not throughput) is the primary objective, the tiling strategy prioritizes **complete layer fitting** in on-chip memory over maximum parallelism — accepting lower DSP utilization to avoid the **100ns+ penalty** of HBM access.

The **weight transfer scheduling** algorithm determines the optimal sequence for loading weight tiles from HBM to URAM based on three parameters per tile: **load time** (HBM→URAM transfer), **execution time** (once loaded), and **URAM usage** (arXiv.org) . Tiles where load time  previous tile's execution time and sufficient URAM is available exhibit **zero pipeline stall**; otherwise, the scheduler inserts wait states or reorders tiles to minimize total stall time. For ML models with predictable layer sizes (standard in trained models), this scheduling can be performed offline and hardcoded into the FPGA control logic.

### 7.3 Weight Storage and Feature Vector Layout

The physical layout of model weights and feature vectors in FPGA memory has profound implications for inference latency, particularly for fully-parallel implementations where every weight must be accessible simultaneously. The standard practice is to store weights in **BRAM with complete array partitioning** (`#pragma HLS ARRAY_PARTITION variable=weights complete`), which distributes the array across multiple BRAM blocks — one per element — enabling parallel read of all weights in a single clock cycle (Xilinx) .

For a fully-connected layer with **N inputs × M outputs**, complete partitioning requires **N×M BRAM blocks**, which quickly exhausts available resources for large layers. The compromise is **cyclic or block partitioning** with a factor that balances parallelism against resource usage:

```
// Cyclic partitioning: factor=4 creates 4 BRAM banks, each holding interleaved elements
#pragma HLS ARRAY_PARTITION variable=weights cyclic factor=4 dim=2
// Block partitioning: factor=4 creates 4 contiguous blocks
#pragma HLS ARRAY_PARTITION variable=weights block factor=4 dim=2
```

For decision tree ensembles, weight storage takes the form of **node tables** where each entry contains (feature_index, threshold, left_child, right_child, leaf_value). The METU thesis framework organizes these as **structure-of-arrays** rather than array-of-structures, enabling parallel access to all fields of a node (OpenMETU) . For XGBoost models, the leaf values are **fixed-point scores** (typically 18–24 bits) that are accumulated across trees; the accumulation buffer requires **ARRAY_PARTITION complete** to enable parallel addition of all tree scores (xelera.io) .

Feature vectors in HFT are typically small (**16–128 features**) and derived from order book state: best

bid/ask prices, depths, spreads, mid-price, order flow imbalance, time-weighted quantities, etc. These features should be stored in **registers** (not BRAM) when possible, using `ARRAY_PARTITION complete` to ensure every feature is available for parallel computation in the first clock cycle (Xilinx) . For neural network inference, the input feature vector is loaded into a **shift register** that feeds the first layer's compute engine, with subsequent layer inputs stored in similarly partitioned buffers.

### 7.4 Minimizing Data Movement Bottlenecks

Data movement — the transfer of weights and activations between memory and compute units — dominates energy consumption and often determines inference latency in FPGA-based ML systems. The fundamental principle for HFT optimization is **keep everything on-chip**: model weights in BRAM/URAM, intermediate activations in registers or distributed RAM, and off-chip memory access limited to initial model loading and result reporting (arXiv.org) .

The **data reuse ratio** — total computation divided by total data movement — is the key metric for evaluating memory efficiency (Preprints) . For a dense matrix multiplication (the core operation in neural networks), the reuse ratio is proportional to the tile size: larger tiles enable more computation per weight load, improving efficiency. However, larger tiles require more on-chip memory, creating a trade-off that must be optimized for each FPGA's BRAM/URAM capacity (arXiv.org) .

The hls4ml `ReuseFactor` parameter directly controls this trade-off: `ReuseFactor=1` implies complete partitioning (maximum parallelism, maximum BRAM usage), while `ReuseFactor=N` time-multiplexes DSP slices across N weight elements (reduced parallelism, reduced BRAM usage) (arXiv.org) . For HFT, the optimal configuration typically uses `ReuseFactor=1` for the first layer (where input features are readily available and latency matters most) and `ReuseFactor=4-8` for deeper layers where the additional latency is amortized across the pipeline.

For multi-layer pipelines, **layer fusion** — combining the activation function of one layer with the matrix multiplication of the next — eliminates the need to write intermediate activations to BRAM and read them back, saving **~2 cycles of latency per fusion** (arXiv.org) . Both hls4ml and FINN apply layer fusion automatically during optimization, producing fused "compute+activate" units that maximize data locality and minimize memory traffic.

## 8. Step-by-Step Implementation Plan

### 8.1 Phase 1: Environment Setup and Hardware Selection

The foundation of a successful FPGA-based HFT ML system begins with selecting the appropriate hardware platform and configuring the development environment. For production tick-to-trade systems, the **AMD Alveo UL3524** is the recommended platform due to its purpose-built ultra-low-latency transceivers, hardened MAC/PCS, and explicit FINN framework support (AMD) . For development and prototyping, the **Alveo U250** or **U280** provide larger fabric and HBM2 capacity at the cost of increased network latency, making them suitable for strategy research and secondary signal generation (VMware Blogs) .

The software toolchain requires **Vitis 2023.1 or later** (including Vitis HLS, Vivado Design Suite, and Vitis AI), **Python 3.9+** with ML frameworks (PyTorch, XGBoost, scikit-learn), and the target FPGA-specific tools: **hls4ml** for neural networks, **Conifer** for BDTs, or **FINN + Brevitas** for quantized networks (indico.global) . The development host should be a Linux workstation (Ubuntu 20.04/22.04 LTS) with **64GB+ RAM** (for synthesis of large designs), **500GB+ SSD storage**, and a **PCIe Gen4×16 slot** for the Alveo card.

For the HFT network infrastructure, the development environment should include: a **10/25GbE switch** with PTP timestamping, a **kernel bypass stack** (DPDK or OpenOnload on the host), and exchange **market data simulator** (e.g., NASDAQ ITCH historical replay) for testing (quantvps.com) . The FPGA card should be installed in a server with **CPU isolation** (reserved cores for trading application), **hugepages configured** (2MB/1GB for DPDK), and **NUMA-aligned** PCIe slots to minimize memory access latency (System Design Interview Roadmap) .

### 8.2 Phase 2: Algorithm Selection and Model Training

Algorithm selection for FPGA HFT inference should prioritize **latency-predictable, resource-efficient models** that capture the relevant market microstructure signals. Based on the suitability analysis in Section 4, the recommended hierarchy is: (1) **XGBoost/LightGBM** as the primary signal generator, (2) **compact MLP** (2–3 layers, 32–128 neurons) for complex non-linear patterns, and (3) **CNN** for order book image-based strategies (xelera.io) . LSTM and Transformer architectures should be reserved for secondary, slower-loop predictions due to their higher latency and resource requirements (My Computer Science and Engineering Department) .

Feature engineering for FPGA deployment emphasizes **integer and fixed-point representations** that map directly to hardware arithmetic. Preferred features include: **price-level differences** (best_bid - best_ask, in integer ticks), **depth imbalances** ((bid_depth - ask_depth) / (bid_depth + ask_depth), as fixed-point), **order flow metrics** (trade_count, cancel_ratio, as integers), and **time-based features** (time_since_last_trade, in microseconds) (OpenMETU) . Features requiring division or logarithms should be pre-computed during training and represented as **lookup tables** in the FPGA, or avoided in favor of equivalent linear approximations.

Model training follows standard ML practices with two FPGA-specific constraints: **quantization-aware training** (if using <8 bit precision) and **latency-aware architecture search** (limiting model size to fit in target FPGA resources). For XGBoost models, the `max_depth` parameter should be **8** (to keep unrolled tree latency under 10 cycles), and `n_estimators` should be **1000** (to fit in available BRAM) (xelera.io) . For neural networks, layer sizes should be chosen such that **weight matrices fit in BRAM/URAM** without off-chip access; a practical limit is ~**1 million INT8 weights** for BRAM-only storage on the UL3524 (infinipoint.ai) .

### 8.3 Phase 3: Quantization and Model Conversion

The quantization phase converts the trained FP32 model to a fixed-point representation suitable for FPGA deployment, using either post-training quantization (PTQ) for INT8 or quantization-aware training (QAT) for lower precision. For XGBoost models, this step is often unnecessary: tree thresholds and leaf scores are naturally represented as fixed-point values, and the Conifer framework handles precision configuration automatically (Github) .

For neural networks, the hls4ml workflow provides automated quantization:

```
import hls4ml
import tensorflow as tf

# Load trained model
model = tf.keras.models.load_model('hft_model.h5')

# Configure FPGA target
config = hls4ml.utils.config_from_keras_model(model, granularity='name')
```

```
config['Model']['Precision'] = 'ap_fixed<16,8>'
config['Model']['ReuseFactor'] = 1
config['Model']['Strategy'] = 'Latency'

# Generate HLS project
hls_model = hls4ml.converters.convert_from_keras_model(
    model, hls_config=config,
    output_dir='hft_hls_project',
    fpga_part='xcu200-fsgd2104-2-e'  # Alveo U250
)

# Verify bit-accurate prediction
y_software = model.predict(X_test)
y_hls = hls_model.predict(X_test)
assert np.allclose(y_software, y_hls, atol=1e-3)

# Synthesize to RTL
hls_model.build(csim=True, synth=True, cosim=True)
```

The `granularity='name'` option enables per-layer precision configuration, allowing the first layer (most sensitive to quantization) to use higher precision (e.g., `ap_fixed<16,8>`) while deeper layers use lower precision (e.g., `ap_fixed<8,4>`) (arXiv.org) . The `Strategy='Latency'` option prioritizes pipeline initiation interval over resource usage, essential for HFT.

For BDT models, the Conifer workflow is even more streamlined:

```
import conifer
import xgboost as xgb

clf = xgb.XGBClassifier().fit(X_train, y_train)
cfg = conifer.backends.xilinxhls.auto_config()
cfg['Precision'] = 'ap_fixed<18,8>'
cfg['XilinxPart'] = 'xcvu47p-fsvh2892-2L-e'  # UL3524

model = conifer.converters.convert_from_xgboost(clf, cfg)
model.compile()  # Verify accuracy
model.build()    # Full synthesis
```

### 8.4 Phase 4: HLS Implementation and Optimization

The HLS implementation phase transforms the converted model into optimized RTL through iterative synthesis and analysis. The key optimization loop involves: (1) synthesizing the design, (2) analyzing the **synthesis report** for latency (II, pipeline depth) and resource usage (DSP, LUT, BRAM), (3) applying pragmas to address bottlenecks, and (4) re-synthesizing until targets are met (Xilinx) .

For HFT inference, the target metrics are: **Initiation Interval (II) = 1** (one inference per clock cycle), **pipeline depth < 100 cycles** (latency < 200ns at 500MHz), and **resource usage < 80%** of available DSP/BRAM (leaving headroom for trading infrastructure). Common bottlenecks and their solutions:

| Bottleneck | Symptom | Solution | Pragma/Directive |
|---|---|---|---|
| Memory port contention | II > 1 due to BRAM read limits | Partition array across multiple BRAMs | `ARRAY_PARTITION complete` |
| DSP shortage | Synthesis fails or uses fabric multipliers | Reduce parallelism (increase reuse factor) | `ReuseFactor=4` (hls4ml) |
| Long combinational path | Timing violation at target frequency | Pipeline the operation, insert registers | `PIPELINE II=1` with `latency` option |
| Loop-carried dependency | Cannot achieve II=1 due to data dependency | Restructure algorithm, use partial unroll | `UNROLL factor=N` + array partitioning |

The Vitis HLS **Schedule Viewer** and **Dataflow Viewer** provide visual feedback on the hardware schedule, enabling identification of bottlenecks that may not be obvious from the C++ code (Xilinx) . For complex designs, **layer-by-layer synthesis** (synthesizing each ML layer as a separate function) enables independent optimization and easier debugging.

### 8.5 Phase 5: Vivado Integration and Bitstream Generation

Once the HLS-generated IP meets latency and resource targets, it is integrated into the complete trading system using Vivado IP Integrator. The integration involves connecting the ML inference IP to: (1) **network interface** (Ethernet MAC/PCS for market data ingress/egress), (2) **order book module** (BRAM-based price ladder updated by market data parser), (3) **feature extraction logic** (computing features from order book state), (4) **trading decision logic** (combining ML prediction with risk checks), and (5) **host interface** (PCIe/DMA for monitoring and control) (www.semidesignjobs.com) .

The ML inference IP typically exposes an **AXI-Stream interface** with `ap_vld` handshaking: the upstream feature extraction module asserts `feature_valid` when new features are available, and the ML IP asserts `prediction_valid` when the inference result is ready (indico.global) . For tick-to-trade latency minimization, the entire pipeline should operate in a **single clock domain** (e.g., 500MHz) with no clock domain crossings on the critical path.

The **constraints file (XDC)** defines timing requirements, pin assignments for network interfaces, and clock configurations. For the UL3524, the GTF transceivers require specific clocking: a **644MHz reference clock** for 10G/25G operation, with jitter attenuators and Stratum-3 clock discipline for deterministic timing (Medium) . The PCIe interface is configured for **Gen4×8** (or Gen3×8 for older hosts) with MSI-X interrupts for host communication.

Bitstream generation (compilation of the complete FPGA design into a configuration file) takes **2–6 hours** for complex designs on the UL3524. The resulting `.xclbin` file is loaded onto the FPGA using the Xilinx Runtime (XRT) or embedded in the board's flash for automatic configuration on power-up (indico.global) .

### 8.6 Phase 6: Testing, Co-Simulation, and Deployment

Verification of FPGA-based ML inference follows a three-tier approach: **software simulation** (C-simulation and C/RTL co-simulation), **hardware emulation** (running on FPGA with synthetic data), and **live market testing** (processing real exchange feeds in a controlled environment) (Xilinx) .

**C-simulation** verifies functional correctness by compiling the HLS C++ code and executing it on the host CPU with the same inputs used for the original Python model. Bit-accurate comparison between Python predictions and C-simulation outputs should show **exact agreement** (for fixed-point) or **<0.1% relative error** (for floating-point emulation) (arXiv.org) .

**C/RTL co-simulation** verifies that the generated RTL produces identical outputs to the C++ reference. This step runs the RTL simulation (using Vivado Simulator or ModelSim) with the same test vectors, comparing cycle-by-cycle outputs (Xilinx) . Co-simulation is **slow** (hours for large designs) but essential for catching synthesis bugs that don't appear in C-simulation.

**Hardware validation** deploys the bitstream on the target FPGA and processes synthetic market data at line rate. Key measurements: **inference latency** (measured with FPGA counters), **throughput** (inferences per second), **resource utilization** (actual vs. estimated), and **determinism** (variance in latency across 1M+ inferences) (indico.global) . The **Integrated Logic Analyzer (ILA)** captures internal signals for debugging timing violations or data corruption.

For production deployment, the FPGA system is integrated into the **co-located trading infrastructure**: direct fiber connection to exchange multicast feeds, **PTP-synchronized timestamping** for latency measurement, and **failover logic** that reverts to software trading if the FPGA detects an error. The STAC-T0 benchmark methodology provides a standardized framework for measuring and reporting tick-to-trade latency under realistic market conditions (A Team Insight) .

## 9. Benchmarks and Real-World Performance

### 9.1 STAC-T0: The Industry Standard for Tick-to-Trade Latency

The **STAC-T0 benchmark**, developed by the Securities Technology Analysis Center (STAC), is the authoritative industry standard for measuring tick-to-trade network-I/O latency (A Team Insight) . The benchmark defines **"actionable latency"** as the time interval between the last bit of inbound data needed to make a trading decision (e.g., a price field in a UDP market data packet) and the first bit of the outbound order message — precisely the metric that determines trading profitability (A Team Insight) .

The benchmark tests multiple scenarios: varying packet sizes (68-byte and 507-byte UDP frames), varying data rates (low/medium/high ingress rates), and different queue models. The most important reported metric is the **minimum actionable latency** — the fastest observed tick-to-trade time under optimal conditions — though **mean, 99th percentile, and maximum latencies** are also reported to characterize determinism (A Team Insight) .

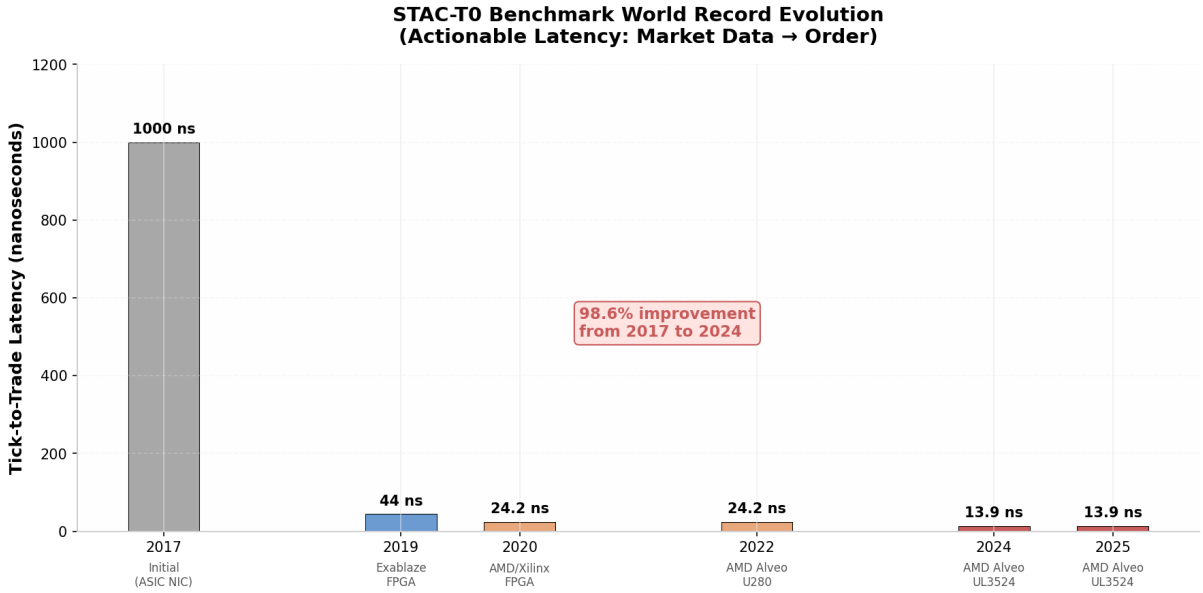


Figure 3: STAC-T0 Benchmark World Record Evolution

The benchmark evolution demonstrates the rapid pace of FPGA technology advancement in HFT. The initial 2017 measurements using ASIC-based NICs reported ~**1 s** latency; by 2019, Exablaze's FPGA-based ExaNIC FDK-XP achieved **31–44ns** (Hedgeweek) . The current world record of **13.9ns** was set in 2024 by AMD and Exegy using the Alveo UL3524 with an **asynchronous combinational logic implementation** of the critical path (A Team Insight) . This **71× improvement** in seven years illustrates why FPGAs have become indispensable for competitive HFT.

### 9.2 ML Inference Latency Benchmarks on Xilinx FPGAs

While STAC-T0 measures the complete tick-to-trade pipeline, dedicated ML inference benchmarks isolate the signal generation component. The following table summarizes measured inference latencies for common HFT ML algorithms on Xilinx hardware:

| Algorithm | Platform | Precision | Latency | Throughput | Source |
|---|---|---|---|---|---|
| **Decision Tree (depth 8)** | Virtex-7 | INT16 | **~10ns** | 100M pred/s | (arXiv.org) |
| **XGBoost (100 trees, depth 6)** | Alveo UL3524 | INT18 | **~80ns** | 12.5M pred/s | (xelera.io) |
| **LightGBM (512K nodes, 128 feat)** | Alveo U250 | INT16 | **1.136 s** (p50) | 880K pred/s | (xelera.io) |
| **MLP (3×64, INT8)** | Zynq US+ | INT8 | **~320ns** | 3.1M pred/s | (Indico) |
| **BNN (binary, small)** | Pynq-Z2 | Binary | **~100ns** | 10M pred/s | (xilinx.github.i |
| **CNN (LOB, INT8)** | Alveo U250 | INT8 | **~500ns** | 2M pred/s | (ntu.edu.tw) |

Table 9 – continued

| Algorithm | Platform | Precision | Latency | Throughput | Source |
|---|---|---|---|---|---|
| **LSTM (3-layer)** | Alveo U55C | FP16 | **1.42 s** | 704K pred/s | (My Computer Science and Engineering Department) |
| **SVM (RBF, 500 SVs)** | Zynq-7000 | FP32 | **~2 s** | 500K pred/s | (IEEE Xplore) |

These benchmarks reveal that **tree-based models** offer the best latency-accuracy trade-off for HFT, with XGBoost ensembles achieving **sub-100ns inference** at accuracy levels competitive with neural networks (xelera.io) . Neural networks provide **higher representational capacity** for complex patterns but at **2–5× higher latency**; they are best deployed when the additional predictive power justifies the latency cost (e.g., in market-making strategies with wider spread capture windows).

### 9.3 Comparative Analysis: FPGA vs. GPU vs. ASIC

The hardware platform landscape for HFT ML inference spans three primary options, each with distinct trade-offs:

**FPGAs** (Alveo UL3524, U280, Versal) offer **reconfigurability** — the ability to update trading algorithms without hardware replacement — combined with **deterministic sub-microsecond latency** and **moderate development cost** ($50K–$500K per deployment) (AMD) . Their primary limitation is **lower peak compute density** than GPUs: while a single GPU can execute thousands of parallel inferences, an FPGA's spatial architecture is optimized for **single-inference latency** rather than batch throughput (linaro.org) .

**GPUs** (NVIDIA A100, H100) excel at **throughput-oriented inference** and **model training**, achieving **<1ms latency** for optimized TensorRT models with batch sizes of 1–8 (Github) . For HFT, GPUs are most effectively deployed in **hybrid architectures**: the FPGA handles tick-to-trade market data processing and simple signal generation, while the GPU runs complex LSTM/Transformer models for position sizing and portfolio optimization on a **slower microsecond-scale loop** (Github) .

**ASICs** (custom silicon) provide the **absolute lowest latency** (potentially <1ns for simple strategies) and **highest power efficiency**, but at **prohibitive development cost** ($10M–$50M NRE) and **zero flexibility** — protocol changes or algorithm updates require new silicon masks (A Team Insight) . Only the largest HFT firms (Citadel, Jump Trading, Virtu) can justify ASIC investment, and even these firms maintain FPGA deployments for strategies requiring frequent algorithm updates (A Team Insight) .

| Metric | FPGA (Alveo UL3524) | GPU (A100 TensorRT) | ASIC (Custom) |
|---|---|---|---|
| **Min Latency** | **13.9ns** (STAC-T0) (A Team Insight) | ~50 s (batch=1) (Github) | **<1ns** (theoretical) |
| **Determinism** | **Excellent** (cycle-accurate) | Poor (kernel launch jitter) | **Excellent** |
| **Reconfigurability** | **Yes** (hours) | N/A (software only) | **No** (new masks) |
| **Development Cost** | $50K–$500K | $10K–$50K | **$10M–$50M** |

Table 10 – continued

| Metric | FPGA (Alveo UL3524) | GPU (A100 TensorRT) | ASIC (Custom) |
|---|---|---|---|
| **Power (Inference)** | **~100W** | ~300W | **~10W** |
| **ML Model Flexibility** | **High** (any algorithm) | Very High (any DL model) | **None** (fixed at tapeout) |
| **Time to Market** | **3–6 months** | 1–2 months | **18–36 months** |

The practical recommendation for most HFT firms is an **FPGA-centric architecture** with GPU augmentation: FPGAs handle all tick-to-trade critical path functions (market data parsing, order book, ML signal generation, order transmission), while GPUs handle model training, backtesting, and secondary analytics. This architecture achieves **99.83% of the latency benefit** of ASICs at **1% of the development cost**, with the flexibility to adapt to changing market conditions and exchange protocols (Github) .

### 9.4 Production Deployment Considerations

Production deployment of FPGA-based ML inference for HFT requires attention to **reliability, monitoring, and regulatory compliance** beyond raw performance. The FPGA system must include **watchdog timers** that detect hung states and trigger failover to software trading, **heartbeat mechanisms** that confirm the ML model is producing predictions within expected bounds, and **circuit breakers** that halt trading if latency exceeds predefined thresholds (Medium) .

**Latency monitoring** at nanosecond precision requires **hardware timestamping**: the FPGA captures the arrival time of market data packets (using PTP-synchronized counters) and the transmission time of order messages, logging these timestamps to on-chip buffers for periodic host retrieval (www.semidesignjobs.com) . The **99th-percentile latency** and **jitter** (standard deviation of latency) are the primary operational metrics, with alerts triggered if either exceeds strategy-specific thresholds.

**Model updating** in production presents a unique challenge: retraining the ML model may improve accuracy, but deploying the updated model requires either (1) **bitstream regeneration** (2–6 hours of synthesis) or (2) **runtime model loading** (using the Conifer FPU or similar programmable IP). The FPU approach enables **sub-millisecond model updates** by writing new tree thresholds and leaf values to the FPGA's configuration registers without interrupting trading (Github) . For neural networks, hls4ml models require bitstream regeneration, so updates are typically scheduled during **market close** or **low-activity periods**.

**Regulatory compliance** (SEC Rule 15c3-5 in the US, MiFID II in Europe) requires **pre-trade risk checks** that verify order limits, position constraints, and price reasonableness before execution. These checks add **~20ns** of latency when implemented in FPGA logic — negligible compared to the total tick-to-trade budget — and are typically integrated into the pipeline between ML signal generation and order transmission (Medium) . The FPGA maintains **audit logs** of all trading decisions with nanosecond timestamps, satisfying regulatory requirements for order traceability.

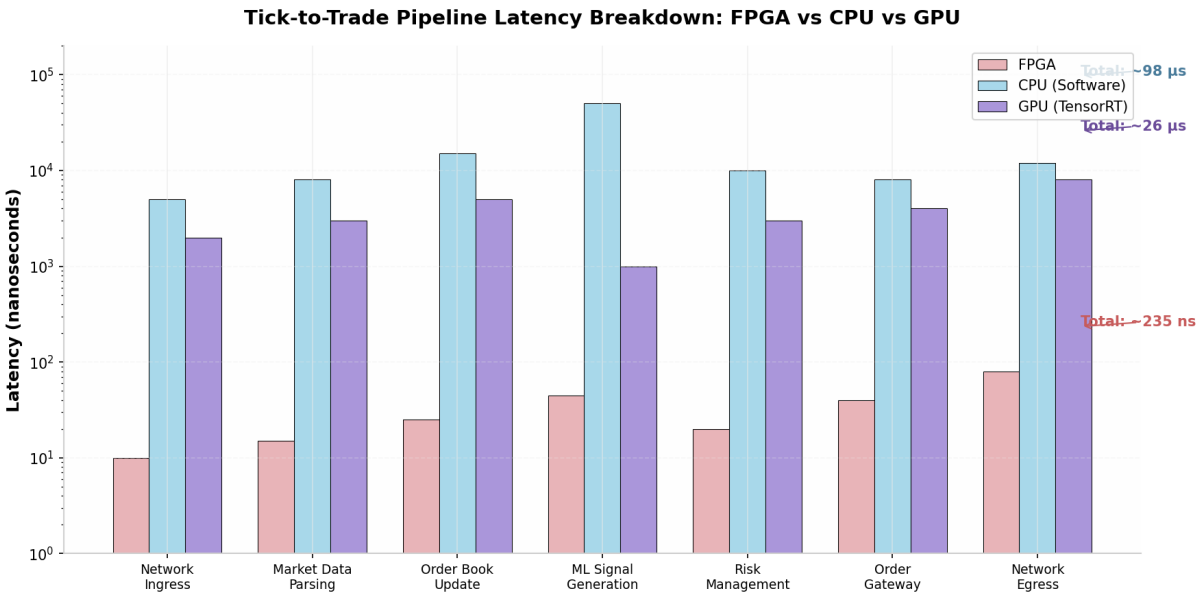


Figure 4: Tick-to-Trade Pipeline Latency Breakdown

## 10. Advanced Topics and Future Directions

### 10.1 Multi-Model Ensemble Architectures

Production HFT systems rarely rely on a single ML model; instead, they deploy **ensembles of heterogeneous models** that combine predictions through weighted voting, stacking, or cascading. A typical architecture might include: (1) a **fast decision tree** (~20ns) for primary signal generation, (2) an **MLP** (~100ns) for pattern confirmation, and (3) an **LSTM** (~1 s) for trend prediction, with the trading decision requiring agreement from at least two models before execution (xelera.io) .

FPGA implementation of multi-model ensembles leverages **pipeline parallelism**: while the LSTM is processing time-series features for the current tick, the decision tree and MLP are processing order book features for the next tick, and the trading decision logic is combining predictions from the previous tick. This **temporal pipelining** enables ensemble inference at the rate of the slowest model (1 s for LSTM) while maintaining tick-to-trade latency of ~**1.2 s** for the complete ensemble (Medium) .

The **Xelera Silva** commercial platform implements this architecture with two deployment modes: **Offload Mode** (PCIe-attached accelerator with 1.0–1.4 s inference latency) and **Inline Mode** (SmartNIC datapath integration with ~**400ns** end-to-end inference by eliminating PCIe transfer) (xelera.io) . The Inline Mode represents the state-of-the-art for production HFT ML inference, achieving sub-microsecond latency for tree-based models with deterministic performance.

### 10.2 Online Learning and Model Adaptation

Financial markets are **non-stationary**: the statistical relationships that predictive models exploit decay over time as market participants adapt and arbitrage away inefficiencies. Online learning — updating model parameters incrementally as new data arrives — is essential for maintaining predictive accuracy, but poses challenges for FPGA deployment where model weights are typically baked into the hardware design.

For decision tree ensembles, **incremental updates** are feasible when using programmable IP like the Conifer FPU: leaf scores can be adjusted based on recent prediction errors, and thresholds can be shifted

to account for changing feature distributions (Github) . These updates require only **register writes** (nanoseconds) rather than bitstream regeneration. For neural networks, **weight perturbation** techniques (adding small random adjustments to weights based on recent performance) can be implemented in FPGA logic, though the limited precision of fixed-point arithmetic constrains the magnitude of valid updates.

The **Versal AI Engine** architecture promises to enable more sophisticated online learning: the AIE array's programmability allows **weight matrices to be updated via DMA** without interrupting inference, and the high-bandwidth NoC enables **gradient accumulation** across multiple inference cycles (Emergent Mind) . While current AIE tooling is immature for this use case, future generations are expected to support **continuous model adaptation** at microsecond timescales.

### 10.3 Hybrid FPGA-GPU Architectures

The optimal HFT ML architecture combines FPGAs and GPUs in a **heterogeneous pipeline** that leverages each platform's strengths. The FPGA handles: (1) **network ingress/egress** (kernel bypass, protocol parsing), (2) **order book maintenance** (sub-microsecond updates), (3) **fast signal generation** (decision trees, compact MLPs), and (4) **pre-trade risk checks** (deterministic compliance). The GPU handles: (1) **complex model inference** (LSTMs, Transformers for trend prediction), (2) **model training** (overnight retraining on historical data), and (3) **portfolio optimization** (mean-variance, risk parity) (Github) .

The communication between FPGA and GPU is critical for overall system performance. **GPUDirect RDMA** enables the FPGA to write inference inputs directly into GPU memory without CPU involvement, achieving **sub-microsecond** FPGA-to-GPU transfer latency (quantvps.com) . Alternatively, **shared pinned memory** (allocated with `cudaHostAlloc`) allows both devices to access the same buffer, with the FPGA producing features and the GPU consuming them in a producer-consumer pattern.

The QuantumEdge open-source platform exemplifies this hybrid architecture: FPGA handles FIX parsing (14ns), order book updates (4ns), and lock-free queue operations (50ns), while GPU runs TensorRT-optimized DQN reinforcement learning inference (<1ms) for strategy selection (Github) . The total end-to-end latency of **350ns** for FPGA-only operations and **<1.5ms** for GPU-augmented decisions demonstrates the viability of this architecture for production HFT.

### 10.4 Emerging Trends: Transformer Quantization and Sparsity

Transformer architectures, despite their current impracticality for sub-microsecond HFT inference, are the subject of active research for **medium-latency trading strategies** (microsecond-to-millisecond timescales) where their superior representational capacity can justify increased latency. Recent advances in **sparse attention** (Sparse Transformer, Longformer, Performer) reduce attention complexity from $O(n^2)$ to $O(n)$, making FPGA implementation more feasible (Gazi University) .

The **FAMOUS** FPGA accelerator for Transformer attention achieves **328 GOPS throughput** on Alveo U250 by using **HLS-based tiling** to decompose large matrix multiplications into URAM-fitting tiles, with a **runtime-programmable scheduler** that adapts to different sequence lengths and embedding dimensions (arXiv.org) . While the ~**10 s latency** is too slow for tick-to-trade, it is viable for **market regime classification** or **volatility forecasting** that operates on millisecond timescales.

**Structural pruning** — removing entire attention heads or feed-forward dimensions rather than individual weights — is particularly effective for Transformer FPGA deployment because it maintains regular memory access patterns (peipeizhou-eecs.github.io) . The algorithm-hardware co-design frame-

work achieves **1.87 TOPS** for sparse Transformer inference by matching the pruning pattern to the FPGA's systolic array dimensions, demonstrating that **sparsity and hardware efficiency can be jointly optimized** (peipeizhou-eecs.github.io) .

### 10.5 Regulatory and Risk Management Implications

The deployment of ML-driven trading strategies on FPGA hardware raises **regulatory and operational risk considerations** that extend beyond technical performance. Regulatory frameworks (SEC Rule 15c3-5, MiFID II Algorithmic Trading requirements) mandate **pre-trade risk controls**, **kill switches**, and **audit trails** — all of which must be implemented in FPGA logic with deterministic latency (Medium) .

**Pre-trade risk checks** on FPGA include: **position limits** (rejecting orders that would exceed maximum position size), **price collars** (rejecting orders outside a reasonable price band), **order rate limits** (throttling orders to prevent runaway algorithms), and **credit checks** (verifying sufficient capital for order execution). These checks add ~**20–50ns** of latency when implemented as combinational logic, and are typically placed **between ML signal generation and order transmission** in the pipeline (Medium) .

**Kill switches** — mechanisms to immediately halt all trading activity — must operate with **guaranteed response time**. FPGA implementations use **dedicated hardware timers** that trigger if no heartbeat is received from the monitoring system within a specified interval (typically 1–10ms), automatically canceling all open orders and disabling new order generation. The deterministic nature of FPGA logic ensures that the kill switch response time is **bounded and predictable**, unlike software implementations where OS scheduling delays could extend response times unpredictably (www.semidesignjobs.com) .

**Model risk** — the risk that the ML model produces incorrect predictions due to distributional shift, adversarial inputs, or training data bias — is mitigated through **ensemble disagreement detection** (halting trading if models disagree beyond a threshold), **prediction confidence thresholds** (only trading when model confidence exceeds a minimum), and **out-of-distribution detection** (identifying market conditions not represented in training data) (xelera.io) . These safeguards add minimal latency (~10ns) when implemented as combinational comparisons in the FPGA decision logic.